How We Built EduVers

This guide tells the story of how EduVers was made, step by step, from an empty folder to a website on the internet. It also shows how the pages are **linked** to each other and to the code that makes them work.

It is written so simply that a Grade 5 student can follow it. If you read `HOW-EDUVERS-WORKS.md` first, this guide will be even easier.

The whole story in one picture

```
 1. Get the tools        →  2. Make an empty project  →  3. Build the tables
 4. Make the cooks       →  5. Write the menu         →  6. Make the waiters
 7. Make the pages       →  8. Link the pages         →  9. Add the guards
10. Add bells and emails → 11. Test everything        → 12. Put it on the internet
```

Each step below explains one box.

Step 1: Get the tools

Before building a house, you need a hammer and nails. Before building a website, you need these programs on the computer:

Tool	What it is like
Laragon	A toolbox that holds PHP and MySQL in one place
PHP	The language the website thinks in
MySQL	The storage room where everything is saved
Composer	A delivery truck that brings PHP parts made by other people
Node.js and npm	Another delivery truck, for the colors and buttons
VS Code	The notebook where we write the code

Step 2: Make an empty project

We typed **one** command in Laragon's Terminal:

```
composer create-project laravel/laravel eduvers
```

This made a new folder called `eduvers`, with all of **Laravel** inside it. Laravel is a ready-made toolbox for websites. It already knows how to show pages, save data and keep users signed in. We only had to add the parts that are special to our school.

Then we told Laravel where the database is. That is written in a settings file called `.env`:

```
DB_DATABASE=eduvers
DB_USERNAME=root
```

Step 3: Build the tables (the storage room)

The database is made of **tables**, like Excel sheets. Each table holds one kind of thing.

To make a table, we write a **migration**, a building plan. Example: the plan for the sections table says "a section has a name, a capacity, a grade level and a strand."

We made a new building plan with this command:

```
php artisan make:migration create_sections_table
```

And we built all the tables with:

```
php artisan migrate
```

The order mattered. You can't put a section in a grade level that doesn't exist yet. So we built the tables like stacking blocks, from the bottom up:

```
           Lessons, Quizzes
                 ▲
       Classes (subject in a section)
                 ▲
       Sections          Subjects
          ▲                 ▲
   Grade levels + Strands + School year
```

How the tables are linked to each other

Tables are linked with **ID numbers**. Every row has its own number, like a student number. When one table needs another, it keeps that number.

This table	keeps the number of	So we know...
`sections`	a grade level and a strand	"ICT 11-A" is **Grade 11**, **ICT**
`users` (students)	a section	Juan is in **ICT 11-A**
`subjects`	a grade level and a strand	"Programming 1" is for **Grade 11 ICT**
`subject_teacher` (a class)	a subject, a section and a teacher	Programming 1 in ICT 11-A is taught by **Ms. Santos**
`lessons`	a subject and a teacher	This lesson belongs to **Programming 1**
`quiz_submissions`	a quiz and a student	Juan got **8 out of 10** on Quiz 1

This is like a family tree. Every child knows who its parent is.

Step 4: Make the cooks (models)

A **model** is the cook who knows one table and its rules. We made one model for each kind of thing:

```
php artisan make:model Section
```

Inside the model, we wrote **how it is linked** to the others:

```
// A section belongs to one grade level
public function gradeLevel() { return $this->belongsTo(GradeLevel::class); }

// A section has many students
public function students() { return $this->hasMany(User::class); }
```

Read it like plain English:

- `belongsTo` means "**belongs to one** ..." (a section belongs to one grade level)
- `hasMany` means "**has many** ..." (a section has many students)

Now, anywhere in the code, we can just write `$section->students` and the cook brings all the students of that section. We don't have to search the storage room by hand.

The cooks also hold the **rules**. For example, the `User` model knows:

> "A student's classes are the classes of their section, plus the classes the admin said yes to."

We wrote that rule **one time**, in the cook. Every page that needs it asks the cook.

Step 5: Write the menu (routes)

Every page of a website has an **address**, like `/student/subjects`. The **menu** (`routes/web.php`) lists every address and says **who handles it**.

```
Route::get('/subjects', [Student\SubjectController::class, 'index'])->name('subjects.index');
```

Read it like a sentence:

> When someone **opens** (`get`) the address **/subjects**, send them to the **SubjectController**, and do its **index** job. Call this page **subjects.index**.

There are two kinds of requests:

Kind	Means	Example
`get`	"**Show** me something"	Open the quiz page

Kind	Means	Example
`post`	"**Save** this for me"	Submit my quiz answers

Grouping the menu by person

We put the addresses in three groups, one for each kind of user:

```
/admin/…    → only admins
/teacher/…  → only teachers
/student/…  → only students
```

So every student page starts with `/student/`, and its name starts with `student.` (like `student.subjects.index`). This keeps the menu tidy.

Step 6: Make the waiters (controllers)

A **controller** is the waiter. It takes the order, asks the cook, and puts the answer on a plate.

```
php artisan make:controller Student/SubjectController
```

Each **job** of the waiter is called a **method** (or **function**). One controller can have many jobs:

Job name	What it does
`index`	Show the **list** (all my subjects)
`show`	Show **one** thing (one subject)
`create`	Show an **empty form**
`store`	**Save** a new thing
`edit`	Show a form to **change** something
`update`	**Save the changes**
`destroy`	**Delete** it

Laravel uses these same seven names everywhere, so every programmer knows what they mean.

Step 7: Make the pages (views)

The **view** is the plate: the page you see. Pages are written in **Blade**, which is plain HTML with little **holes** for data.

```
<h1>My subjects</h1>

@foreach ($classes as $class)
    <p>{{ $class->subject->name }}</p>
@endforeach
```

- `{{ ... }}` means "**put the value here**."
- `@foreach` means "**do this again for each one**." Five subjects make five lines.

All pages share one **layout**: the frame with the logo, the side menu and the clock. Each page only fills in the middle part. That is why every page looks the same.

Step 8: Link the pages to each other ⭐

This is how one page leads to another, and how a button reaches the right function.

Links use the page's name, not its address

Remember that every page on the menu has a **name**, like `student.subjects.show`. To link to a page, we use its name:

```
<a href="{{ route('student.subjects.show', $subject) }}">Open subject</a>
```

`route(...)` looks up the address for that name. If we ever change the address, every link still works, because the name stays the same.

The full chain, from click to code

Here is how **one click** travels through everything we built:

```
You click a link
       |
       ▼
route('student.subjects.show')      ← the link uses the page's NAME
       |
       ▼
/student/subjects/5                 ← the name becomes an ADDRESS
       |
       ▼
routes/web.php                      ← the MENU finds who handles it
       |
       ▼
SubjectController → show()          ← the WAITER and its JOB (function)
       |
       ▼
Subject model                       ← the COOK gets the data
       |
       ▼
Database                            ← the STORAGE ROOM
       |
       ▼
student/subjects/show.blade.php     ← the PLATE (page)
       |
       ▼
You see the subject page
```

Buttons that save something (forms)

A button that **saves** (like "Submit quiz") is a **form**. It sends a `post` request:

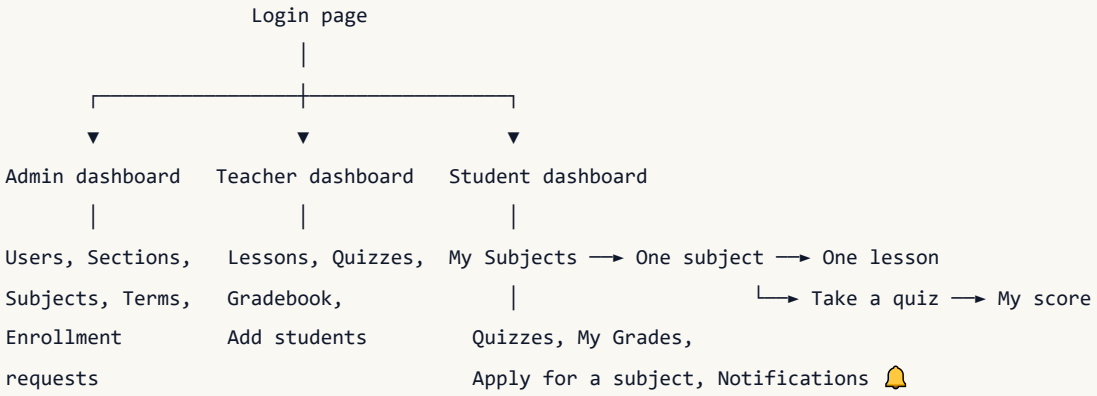
```
<form method="POST" action="{{ route('student.quizzes.submit', $quiz) }}">
    @csrf
    … the questions …
    <button>Submit</button>
</form>
```

- `action` tells the form **which page name** to send the answers to.
- `@csrf` is a **secret ticket**. It proves the form really came from our website, so other websites can't send fake answers.

The side menu

The side menu on the left is a list of links. Each role gets its own list. A student sees "My Subjects", "Quizzes" and "My Grades". A teacher sees "Lessons", "Quizzes" and "Gradebook". It is written once, in the layout, so it shows on every page.

A map of how the main pages link



Step 9: Add the guards (safety)

Now that the pages work, we made sure only the **right people** can open them.

The login. We wrote our own login page instead of using a ready-made one, so it looks like the school. When you sign in, EduVers checks your email and password, then sends you to **your own** dashboard.

The guard at the door (middleware). Before a page opens, the guard checks:

1. Are you signed in? (`auth`)
2. Are you the right kind of user? (`role:student` , `role:teacher` , ...)

We put the guard on the whole group at once:

```
Route::middleware('role:student')->prefix('student')->group(function () {
    // every student page is in here
});
```

So **every** student page is protected, and we can't forget one.

Ownership rules (policies). A teacher can edit **their own** lessons, but not another teacher's. These rules are in `app/Policies/` .

More safety we added:

- 5 wrong passwords → the login waits before letting you try again.
- 30 minutes of doing nothing → you are signed out by yourself.
- Forgot password → an email link that works one time, for 60 minutes.
- Passwords are scrambled before saving. Nobody can read them.

Step 10: Add the bell and the emails

When a teacher posts a lesson, the students should know. So we added **notifications**:

1. The teacher clicks **Save** on a new lesson.

2. EduVers finds every student in that class.
3. Each student gets a message in the **bell** 🔔 right away.
4. Each student also gets an **email**.

The bell checks for new messages every **15 seconds**, so the student sees it without refreshing the page. The emails are sent by **Brevo**, an email service.

Step 11: Test everything

A **test** is a small program that uses the website like a person would, and checks the answer. Example test, in plain words:

> "Make a teacher. Make a student. Try to open the teacher's page as the student. The answer **must** be **403 Forbidden**."

We wrote tests for logins, roles, quizzes, enrollment and notifications. We run them all with one command:

```
php artisan test
```

✅ Green = everything works. ❌ Red = something broke, and the test tells us where.

Step 12: Put it on the internet

A website on your own computer only works for you. To let everyone use it, we put it on the internet:

Service	Its job	Like...
GitHub	Keeps a copy of all the code online	A safe locker for the code
Railway	Runs the website on the internet	A computer that is always on
TiDB Cloud	Keeps the database on the internet	An online storage room
Brevo	Sends the emails	A post office

How it works:

1. We save the code to **GitHub**.
2. **Railway** sees the new code and builds the website by itself.
3. When it starts, it builds any new tables in **TiDB Cloud**.
4. The website is online. Anyone with the link can open it.

The passwords for the database and email are **not** written in the code. They are kept safely in Railway's settings.

The online database: TiDB Cloud

The online storage room is on **TiDB Cloud** (tidbcloud.com). TiDB speaks the same language as MySQL, so EduVers works with it **without changing any code**. Only the settings are different.

What we made there:

Thing	Our name	Simple meaning
Project	`eduvers-db`	A folder that holds our database
Cluster (the database server)	`eduvers-db`	The online computer that keeps the data
Plan	**Starter**	TiDB's **free** plan
Database	`eduvers`	The storage room EduVers uses (same name as on Laragon)

How we set it up, step by step:

1. Make a free account at **tidbcloud.com**.
2. Click **Create Resource** and choose the **Starter** (free) plan. Name it `eduvers-db`.
3. When its status shows **Active** 🟢, click it, then click **Connect**.
4. Copy the **host**, **port** (4000), **username** and **password** into Railway's settings (`DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD`).
5. Deploy the website. When Railway starts it, EduVers builds **all the tables by itself** (`RUN_MIGRATIONS=true`), the same way `php artisan migrate` does on Laragon.

How to look inside the online database:

1. In TiDB Cloud, click **eduvers-db**, then **SQL Editor** on the left.
2. On the left, open **eduvers**. You will see all the tables:

   `academic_terms`, `announcements`, `cache`, `cache_locks`, `class_enrollments`, `failed_jobs`, `grade_levels`, `job_batches`, `jobs`, `lessons`, `migrations`, `notifications`, `password_reset_tokens`, `quiz_questions`, `quiz_section`, `quiz_submissions`, `quizzes`, `sections`, `sessions`, `subject_teacher`, `subjects`, `track_strands`, `users`

   These are the **same tables** as on Laragon, because both were built from the same migration files.
3. Type a question in the editor, then click **Run** (or press **Ctrl + Enter**). For example, to see every account:

```
USE eduvers;
SELECT id, name, email, role, created_at FROM users ORDER BY id;
```

`USE eduvers;` means "open the eduvers storage room." `SELECT … FROM users` means "show me these columns from the users table."

⚠️ **Only read** in the SQL Editor (`SELECT`). Commands like `DELETE`, `UPDATE` or `DROP` change the **real** data that teachers and students use, and they cannot be undone. Make changes through the EduVers website instead.

Some tables are Laravel's own helpers, not school data: `cache`, `cache_locks`, `sessions` (who is signed in), `jobs`, `job_batches`, `failed_jobs` (waiting tasks), `migrations` (which building plans already ran) and `password_reset_tokens`.

The settings (variables) we put on Railway

Railway → the **eduvers** service → **Variables** → **Raw Editor**. Paste this list, then replace every `<…>` with your own value. The same list is in `deploy/railway.env.example`.

🔒 **The 5 secrets** (`APP_KEY`, `BREVO_API_KEY`, `DB_HOST`, `DB_USERNAME`, `DB_PASSWORD`) are **never** written in any file or document. Anyone who has them can open your database or send emails from your account. Keep them only in Railway.

```
# The website
APP_NAME=EduVers
APP_ENV=production
APP_DEBUG=false
APP_KEY=<run: php artisan key:generate --show>
APP_URL=https://<your-website-address>
APP_TIMEZONE=Asia/Manila
PORT=8080

# The database (TiDB Cloud)
DB_CONNECTION=mysql
DB_HOST=<from TiDB Cloud → Connect, e.g. gateway01.ap-southeast-1.prod.aws.tidbcloud.com>
DB_PORT=4000
DB_DATABASE=eduvers
DB_USERNAME=<from TiDB Cloud → Connect, looks like xxxxxxxx.root>
DB_PASSWORD=<from TiDB Cloud → Connect>
MYSQL_ATTR_SSL_CA=/etc/ssl/certs/ca-certificates.crt

# Emails (Brevo)
MAIL_MAILER=brevo
BREVO_API_KEY=<from Brevo → SMTP & API → API Keys>
MAIL_FROM_ADDRESS=<an email you verified in Brevo>
MAIL_FROM_NAME=EduVers
NOTIFICATIONS_MAIL=true

# How the server runs
CACHE_STORE=database
SESSION_DRIVER=database
SESSION_SECURE_COOKIE=true
QUEUE_CONNECTION=sync
FILESYSTEM_DISK=local
LOG_CHANNEL=stderr
LOG_LEVEL=warning
RUN_MIGRATIONS=true
RUN_QUEUE_WORKER=true
SEED_DEMO_DATA=false
```

What each group means, in simple words:

Setting	Simple meaning
`APP_DEBUG=false`	Don't show error details to visitors (they could help a hacker)
`APP_KEY`	The website's secret lock key. Sign-ins and passwords depend on it
`APP_URL`	The website's address, used in links inside emails

Setting	Simple meaning
`PORT=8080`	The door number the website listens on inside Railway
`DB_…`	Where the online storage room (TiDB Cloud) is, and how to get in
`MYSQL_ATTR_SSL_CA`	TiDB only talks over a **locked (encrypted) line**. This is the key file for it
`MAIL_MAILER=brevo`	Send emails through Brevo's website, because Railway blocks normal email
`QUEUE_CONNECTION=sync`	Send each email right away, while the page saves
`SESSION_SECURE_COOKIE=true`	The sign-in only works over the safe `https://` address
`RUN_MIGRATIONS=true`	Build any new tables automatically each time the server starts
`SEED_DEMO_DATA=false`	**Don't** add the demo accounts on the real website

When the free trial ends

Railway's free **trial** lasts 30 days. After that, the website stops unless someone pays or moves it. What happens to each part:

Part	Free after the trial?
TiDB Cloud (database)	Has a free plan. The data stays.
Brevo (emails)	Has a free plan (about 300 emails a day).
GitHub (code)	Free.
Railway (the website)	**Not free after the trial.** The website must move or be paid for.
Laragon (on your own computer)	Free forever. Best for the defense: it works even without internet.

To move the website to another host, you only need **the same list of settings above**. The code doesn't change, because everything that is different between computers lives in these settings. Check the host's current prices first, because free plans change often. Also check two things: whether the free plan **sleeps** when nobody visits (the first visit is then slow), and whether **uploaded files** (lesson attachments, videos) are kept after a restart. If they aren't, uploads will disappear.

The File Finder: where every part lives 🔎

If a panelist says **"Show us the file"**, use this list.

How to open a file fast in VS Code:

1. Press **Ctrl + P**, type the file name (for example `User.php`), press **Enter**.
2. Press **Ctrl + F**, type the function name (for example `studentClasses`), press **Enter**.

The guards (who can open what)

If they ask about…	Open this file	Look for
Where the **role is checked** (admin, teacher, student)	`app/Http/Middleware/EnsureUserHasRole.php`	`handle()`
Where each **page group gets its guard**	`routes/web.php`	`Route::middleware('role:student')` and the other groups
The **list of roles**	`app/Enums/UserRole.php`	`Admin`, `Teacher`, `Student`
Teachers can only edit their own lessons / quizzes	`app/Policies/LessonPolicy.php` and `app/Policies/QuizPolicy.php`	`update()`
Who may **take a quiz**	`app/Policies/QuizPolicy.php`	`take()`

Login and safety

If they ask about…	Open this file	Look for
The **login** and the **5 wrong tries** lock	`app/Http/Controllers/Auth/LoginController.php`	`MAX_ATTEMPTS = 5` and `store()`
The **login page** design	`resources/views/auth/login.blade.php`	
Auto sign-out after 30 minutes	`app/Http/Middleware/LogoutIdleUsers.php`	`handle()`
Where the **30 minutes** is set	`config/eduvers.php`	`idle_minutes`
Forgot password	`app/Http/Controllers/Auth/PasswordResetController.php`	`store()` and `update()`
The **reset password email**	`app/Providers/AppServiceProvider.php`	`ResetPassword::toMailUsing`
Passwords are scrambled (hashed)	`app/Models/User.php`	`'password' => 'hashed'`
Private lesson files	`app/Http/Controllers/LessonAttachmentController.php`	`Gate::authorize('view', $lesson)`

Students, classes and subjects

If they ask about…	Open this file	Look for
Which **subjects a student has**	`app/Models/User.php`	`studentClasses()`
Putting a student in a section	`app/Models/User.php`	`enrollInSection()`
The **admin enrollment page**	`app/Http/Controllers/Admin/StudentEnrollmentController.php`	
Which sections can take a subject	`app/Models/Subject.php`	`eligibleSections()`
Classes made automatically for a new subject or section	`app/Models/Subject.php` and `app/Models/Section.php`	`syncClasses()`
What a **class** is (subject × section × teacher)	`app/Models/SubjectTeacher.php`	
The **"My Subjects" page**	`app/Http/Controllers/Student/SubjectController.php`	`index()` and `show()`
A subject with lessons **can't be deleted**	`app/Http/Controllers/Admin/SubjectController.php`	`destroy()`

Joining a class from another section

If they ask about…	Open this file	Look for
The **request** and the **approve / reject** rule	`app/Models/ClassEnrollment.php`	`request()` and `decide()`
A **student applies**	`app/Http/Controllers/Student/EnrollmentController.php`	`store()`
A **teacher adds a student**	`app/Http/Controllers/Teacher/EnrollmentController.php`	`store()`
The **admin approves**	`app/Http/Controllers/Admin/EnrollmentController.php`	`update()`

Lessons, quizzes and grades

If they ask about…	Open this file	Look for
A teacher **posts a lesson**	`app/Http/Controllers/Teacher/LessonController.php`	`store()`
A teacher **makes and publishes a quiz**	`app/Http/Controllers/Teacher/QuizController.php`	`store()` and `publish()`

If they ask about…	Open this file	Look for
How a **quiz is scored**	`app/Models/Quiz.php`	`submitFor()`
Which **quizzes a student can see**	`app/Models/Quiz.php`	`scopeVisibleTo()`
A student **submits a quiz**	`app/Http/Controllers/Student/QuizController.php`	`submit()`
Only one try per quiz	`database/migrations/2026_09_27_000012_create_quiz_submissions_table.php`	`unique(['quiz_id', 'student_id'])`
The **gradebook**	`app/Support/Gradebook.php`	`build()`
Download grades (CSV / print)	`app/Http/Controllers/Teacher/GradebookController.php` and `app/Support/Csv.php`	`export()` and `print()`

The bell and the emails

If they ask about…	Open this file	Look for
Who gets notified	`app/Support/StudentNotifier.php`	`lessonPosted()` and `quizPublished()`
The **message itself**	`app/Notifications/` (one file per kind of message)	
The **bell** 🔔	`resources/views/components/notification-bell.blade.php`	
The bell **checks every 15 seconds**	`resources/js/app.js`	`setInterval(refresh`
The **email design**	`resources/views/mail/student-activity.blade.php`	
Sending emails with Brevo	`app/Mail/Transport/BrevoApiTransport.php`	

Pages, database and tests

If they ask about…	Open this file	Look for
Every page address	`routes/web.php`	
The **side menu** and page frame	`resources/views/components/layouts/app.blade.php`	`$nav = match`

If they ask about...	Open this file	Look for
The **homepage boxes** (Senior High, College, Tracks)	`app/Support/SchoolProfile.php`	`tiles()`
How the **tables were built**	`database/migrations/` (oldest file first)	
The **sample data and demo accounts**	`database/seeders/EduVersSeeder.php`	
The **tests**	`tests/Feature/` (for example `RoleAccessTest.php`)	

Putting it online

If they ask about...	Open this file
How the **online server is built**	`Dockerfile`
What happens when the **server starts** (tables, emails, website)	`docker/entrypoint.sh`
The **settings needed on Railway**	`deploy/railway.env.example`

Answers to the tricky questions

- **"Show us where the role is checked."** → `app/Http/Middleware/EnsureUserHasRole.php`, then show the groups in `routes/web.php` that use it, like `Route::middleware('role:student')`.
- **"Which code decides a student's subjects?"** → `app/Models/User.php`, function `studentClasses()`. Read it out loud: "*their section's classes, plus the classes the admin approved.*"
- **"What breaks if you delete the `subject_teacher` table?"** → Every class is in that table. Students would have **no subjects**, and teachers would have **no classes**. See `app/Models/SubjectTeacher.php`.
- **"Why is a migration never edited after it has run?"** → Open `database/migrations/`. Laravel remembers which ones already ran and **never runs them again**. A change in an old file would be skipped on every computer that already has the table. So we always write a **new** migration.
- **"Turn off the internet. What still works?"** → The local version on Laragon still runs, because PHP and MySQL are on the computer. Only the **emails** stop, because Brevo is on the internet.

A short summary to say in your defense

> We built EduVers with **Laravel**. First we planned the **tables** in the database and linked them with ID numbers. Then we made a **model** for each table, which knows how it is linked to the others. We listed every page in the **routes**, grouped by admin, teacher and student, and protected each group with a **guard** that checks the user's role. Each page has a **controller** that gets the data from the model and shows it in a **Blade view**. Pages link to each other using **route names**, and forms save data with a **POST** request protected by a **CSRF** token. We added **notifications** by bell and email, wrote **tests** to prove it works, and put it online on **Railway** with the database on **TiDB Cloud**.

Words to remember

Word	Simple meaning
Migration	A building plan for a table
Model	The cook who knows one table and its rules
Relationship	How tables are linked: "belongs to" and "has many"
Route	A page address on the menu, and who handles it
Route name	A nickname for a page, used to make links
Controller	The waiter who handles a page
Method / function	One job of the waiter, like `index` or `store`
View / Blade	The page you see, with holes for data
Layout	The frame shared by every page (logo, menu, clock)
GET	"Show me something"
POST	"Save this for me"
Form	A box with fields and a button that saves data
CSRF token	A secret ticket that proves a form is really from our site
Middleware	The guard that checks you before a page opens
Policy	A rule about who may change what
Notification	A message in the bell 🔔 or by email
Test	A check that the code still works
Deploy	Putting the website on the internet